Task 7 – Build a Module

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task07)

1. Complete the Code – TODO 1 and TODO 2 (milkrate.py)

milkrate.py

```
"""Rate card of the village milk co-operative (used as a module)."""
BASE_RATE = 30.00       # Rs per litre for milk with 3.5 % fat and 8.5 % SNF
FAT_STEP_VALUE = 0.40   # Rs per litre for every 0.1 % of fat above or below 3.5
SNF_STEP_VALUE = 0.30   # Rs per litre for every 0.1 % of SNF above or below 8.5
STD_FAT = 3.5
STD_SNF = 8.5
STEP = 0.1


def rate_per_litre(fat, snf):
    """Returns the price per litre, rounded to 2 decimals."""
    fat_steps = round((fat - STD_FAT) / STEP)
    snf_steps = round((snf - STD_SNF) / STEP)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)


def payout(litres, fat, snf):
    """Returns the amount payable to the farmer, rounded to 2 decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)


if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate, pay = rate_per_litre(fat, snf), payout(litres, fat, snf)
        print(f"{litres:5} L  fat {fat}  snf {snf}  rate {rate:6.2f}  pay {pay:8.2f}")
```

python milkrate.py (identical to the expected output)

```
   10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
  8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
   12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

2. Using the module from a separate script

use_milkrate.py

```
import sys
import milkrate
from milkrate import payout
import milkrate as mr

print("module name   :", milkrate.__name__)
print("payout(8.5, 4.2, 8.7) =", payout(8.5, 4.2, 8.7))
print("mr.rate_per_litre(3.1, 8.3) =", mr.rate_per_litre(3.1, 8.3))
print("loaded once   :", "milkrate" in sys.modules)
```

python use_milkrate.py – note that no table is printed

```
module name   : milkrate
payout(8.5, 4.2, 8.7) = 283.9
mr.rate_per_litre(3.1, 8.3) = 27.8
loaded once   : True
```

Why the table is not printed on import: when a file is run directly Python sets its __name__ to "__main__", so the block under `if __name__ == "__main__":` runs. When the same file is *imported*, __name__ is the module's file name, "milkrate" (printed above), the condition is False and the demonstration is skipped. The module therefore provides only definitions to its importer.

3. The name test demonstrated without the guard

A copy of the module without the guard, and a script importing it

```
"""Same module WITHOUT the __name__ test - used only to demonstrate the problem."""
def rate_per_litre(fat, snf):
    return round(30.00 + round((fat - 3.5) / 0.1) * 0.40 + round((snf - 8.5) / 0.1) * 0.30, 2)

for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7)]:
    print("demo row:", litres, fat, snf, rate_per_litre(fat, snf))

# use_noguard.py:
import milkrate_noguard
print("rate =", milkrate_noguard.rate_per_litre(4.2, 8.7))
```

Output: the demo rows appear merely because the module was imported

```
demo row: 10 3.5 8.5 30.0
demo row: 8.5 4.2 8.7 33.4
rate = 33.4
```

4. Three ways to import

Statement	Name that becomes available	Call written as	Use when
import milkrate	milkrate	milkrate.payout(...)	Clear where a function comes from; no name clashes.
from milkrate import payout	payout (only)	payout(...)	Short code when only one or two names are needed; risk of clashing with another payout.
import milkrate as mr	mr	mr.payout(...)	A shorter alias for a long module name (as in import numpy as np).

5. Standard modules and where Python looks for a module

math gives floor, ceil, isclose; **decimal** gives exact decimal arithmetic (Task 2); **time**/**timeit** give timing (Task 10); **json** reads the rate card (Task 11). When `import milkrate` is executed Python searches, in order: modules already loaded (`sys.modules`), built-in modules, then each folder in `sys.path`: first the folder of the running script (shown as '' / the script folder), then the standard library folders and site-packages. That is why use_milkrate.py works only when milkrate.py is in the same folder (or its folder has been added to sys.path).

Alternative Concept and Complexity Analysis

Alternative: copy rate_per_litre() and payout() into every script that needs them.

Measured import cost

```
sys.path[0:2] = ['', '/usr/lib/python312.zip']
first import 152.4 us, later imports 114 ns each
```

	Module (import)	Copy into every script
Work when a rate changes	edit 1 file; every script sees the change on its next run	edit every copy (k scripts = k edits); a forgotten copy silently pays the old rate – the original volunteer's problem
One-time cost	first import ≈ 150 µs to find, compile (or load the cached .pyc) and run the file	none
Repeated cost	later `import` statements only look in sys.modules: ≈ 0.1 µs	none
Space	one module object in memory, shared by all importers: O(1)	k copies of the code: O(k) text and k copies of the function objects

The cost of importing is paid once per program run and is negligible beside the time to process even 300 entries; the maintenance saving is large. The module is therefore the better design.

Evidence

A working module imported by a separate script, with the __name__ test explained by demonstration. Source: Task07/milkrate.py, use_milkrate.py, milkrate_noguard.py, use_noguard.py.